Text Representations: TF-IDF

Natural Language Processing

Daniel Ortiz Martínez

Table of Contents

Motivation

Bag of Words Representation

We represent a document as a **frequency vector**:

$$\mathrm{x} = \left[\mathrm{count}(w_1),\ \mathrm{count}(w_2),\ \ldots,\ \mathrm{count}(w_V)\right]$$

What works well:

- Simple and interpretable
- Effective for topic classification

Example:

"the cat sat on the mat"

Token	Count
the	2
cat	1
sat	1
on	1
mat	1

The Problem with Frequent Words

The problem with very frequent words

Words that are very frequent across all documents carry little discriminative value

Words with high BoW weight:

- "the", "a", "is", "of" ...
- Appear in *every* document
- Do not help distinguish topics

Truly informative words:

- "neural", "election", "asteroid"
- Only in a few documents
- High discriminative power

What do we Want from a Good Representation?

What do we Want from a Good Representation?

1. Give **more weight** to words that are frequent *in this document*

What do we Want from a Good Representation?

1. Give **more weight** to words that are frequent *in this document*

2. Give **less weight** to words that are frequent *across all documents*

What do we Want from a Good Representation?

1. Give **more weight** to words that are frequent *in this document*
2. Give **less weight** to words that are frequent *across all documents*
3. **Comparable** results across documents of different lengths

Term Frequency (TF)

Term Frequency: the idea

Intuition

If a word appears many times in a document, it is probably relevant to that document.

The simplest version (raw frequency):

$$\mathrm{TF}(t, d) = \frac{\text{number of times } t \text{ appears in } d}{\text{total number of tokens in } d} = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

Example: In a 200-word article, "python" appears 10 times:

$$\mathrm{TF}(\text{“python”}, d) = \frac{10}{200} = 0.05$$

TF variants

Variant	Formula
Raw count	$f_{t,d}$
Normalized	$\frac{f_{t,d}}{\max_{t'} f_{t',d}}$
Log-scaled	$\log(1 + f_{t,d})$

Why log-scaled?

Relevance does not grow linearly with frequency. The 10th occurrence of "python" adds much less information than the 1st.

Inverse Document Frequency (IDF)

Inverse Document Frequency: key idea

Intuition

If a word appears in many documents, it is not very informative.
We should **penalize** it.

$$\mathrm{IDF}(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|} = \log \frac{N}{\mathrm{df}_t}$$

where N is the total number of documents and the denominator is how many documents contain term t

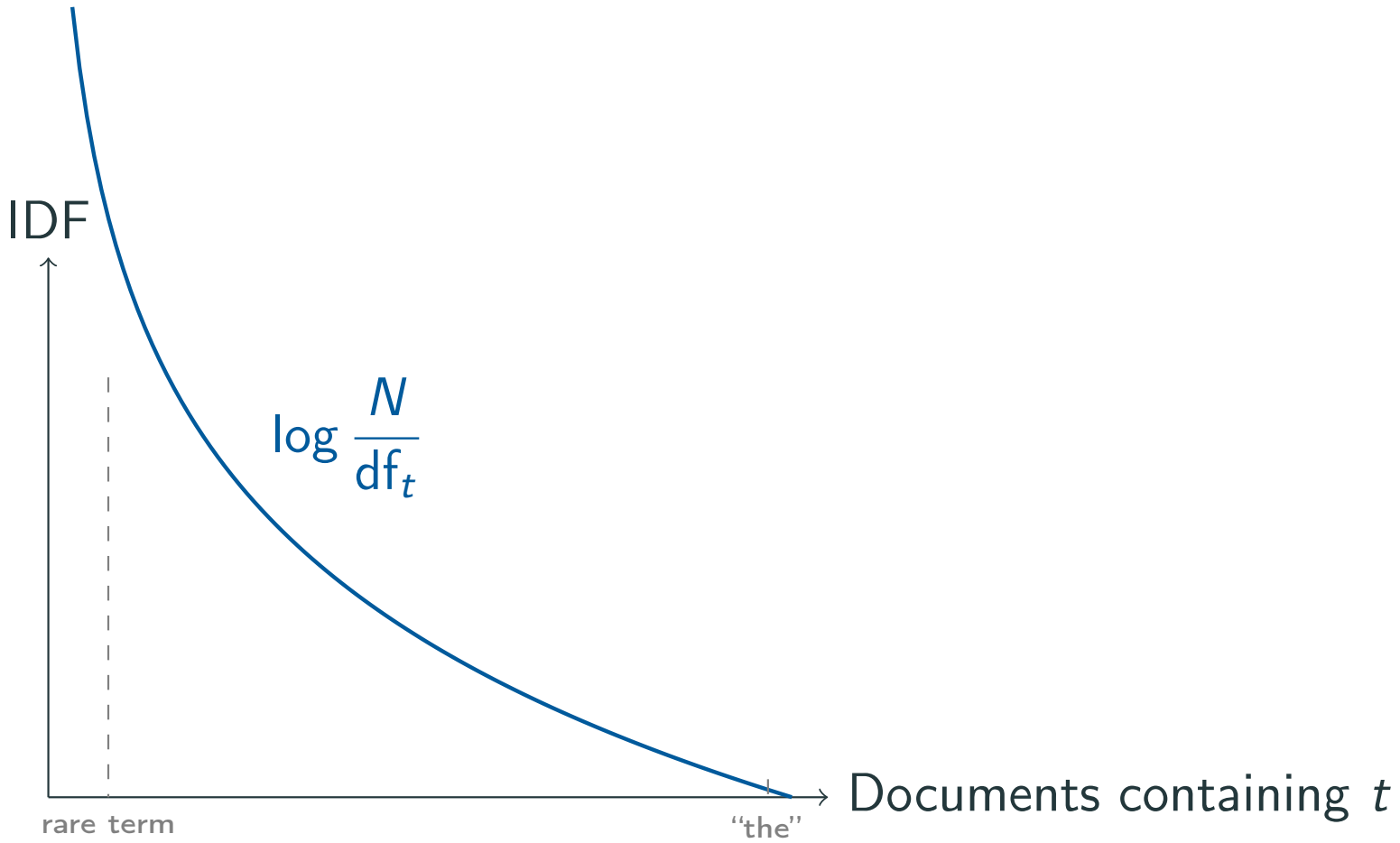
"**the**" appears in *all* docs:

$$\mathrm{IDF} = \log \frac{N}{N} = 0$$

"**asteroid**" appears in 5 of 1000:

$$\mathrm{IDF} = \log \frac{1000}{5} \approx 5.3$$

IDF: behaviour



Higher corpus presence $\Rightarrow$ lower IDF $\Rightarrow$ less weight in the final representation

IDF smoothing

Problem: if a term never appears in the corpus, $\mathrm{df}_t = 0 \Rightarrow$ division by zero

Standard solution:

$$\mathrm{IDF}(t) = \log \frac{1 + N}{1 + \mathrm{df}_t} + 1$$

- The $+1$ in numerator and denominator avoids division by zero
- The trailing $+1$ ensures IDF ≥ 1 (avoids zero weights for those terms appearing in all documents)

TF-IDF: full formula

Combining TF and IDF

Definition

$$\mathrm{TFIDF}(t, d, D) = \mathrm{TF}(t, d) \times \mathrm{IDF}(t, D)$$

Term	TF (in d)	IDF (in D)	TF-IDF	Interpretation
"the"	high	≈ 0	low	irrelevant
"python"	high	high	high	very informative
"asteroid"	low	high	medium	somewhat informative

L2 Normalization of the Final Vector

Two documents about the same topic but of different length will have TF-IDF values at different scales.

Solution: normalize the resulting vector to unit norm:

$$\tilde{x}_d = \frac{x_d}{\|x_d\|_2}$$

- All documents lie on the unit hypersphere
- Cosine similarity between two documents equals their dot product

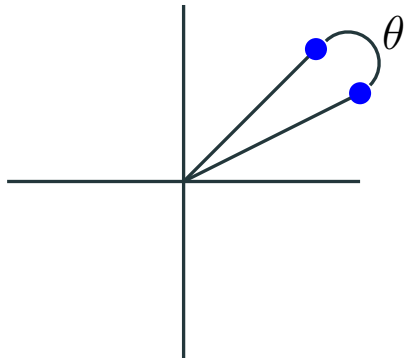
Cosine Similarity

- The cosine of two non-zero n-dimensional vectors can be derived from the Euclidean dot product:

$$p^T \cdot q = ||p||\,||q|| \cos(\theta)$$

- The cosine similarity, cs, is then given by:

$$\mathrm{cs}(p, q) = \cos(\theta) = \frac{p^T \cdot q}{||p||\,||q||} = \frac{\sum_{i=1}^{D} p_i \cdot q_i}{\sqrt{\sum_{i=1}^{D} p_i^2} \cdot \sqrt{\sum_{i=1}^{D} q_i^2}}$$



- $\mathrm{cs}(p, q) = 1 \rightarrow$ very similar documents and $\mathrm{cs}(p, q) = 0 \rightarrow$ unrelated documents

Cosine Similarity

With L2-normalized vectors:

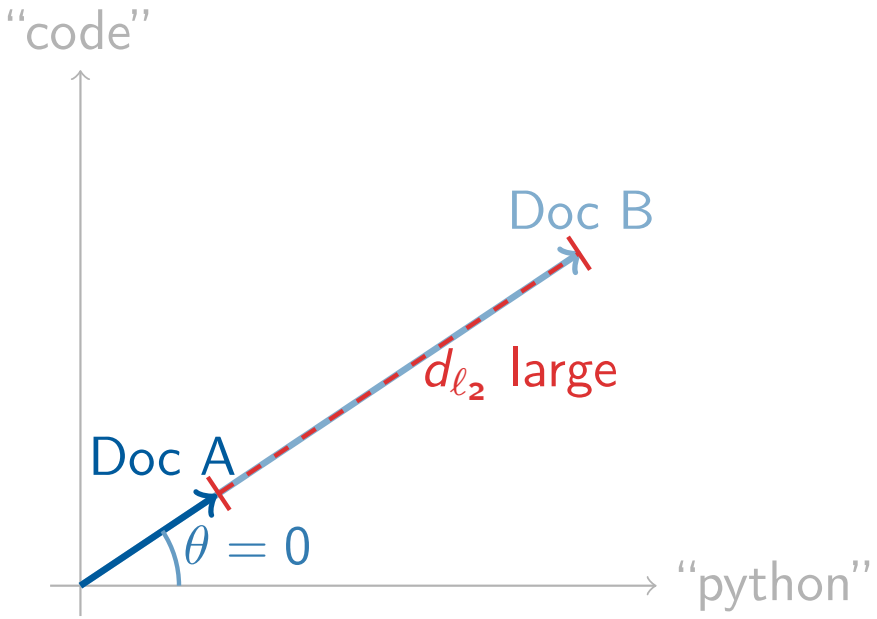
$$\cos(\theta) = p^T \cdot q$$

Similarity reduces to a dot product, which is cheap to compute at scale

Why normalize? Cosine vs Euclidean on Raw Counts

Two documents on **the same topic**, different lengths:

	Doc A	Doc B
Length	50 words	500 words
"python"	3	30
"code"	2	20
"function"	1	10



Why normalize? Cosine vs Euclidean on Raw Counts

Euclidean distance on raw counts

Doc B has $10\times$ larger values $\Rightarrow$ large d_{ℓ_2}, even though both documents discuss *exactly the same topic*.

Cosine similarity on raw counts

Both vectors point in the **same direction** $\Rightarrow \cos(0) = 1$, correctly identifying them as identical in content.

After L2 normalization, Euclidean distance and cosine similarity become equivalent

TF-IDF in scikit-learn

TfidfVectorizer: Basic Usage

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "the cat ate the fish",
    "the dog ate the meat",
    "the cat and the dog are friends",
]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

print(X.shape)                  # (3, vocabulary_size)
print(vectorizer.vocabulary_)
print(X.toarray())
```

Key parameters

Parameter	Default	Description
`max_features`	None	Cap vocabulary size
`min_df`	1	Minimum document frequency (or fraction) to include a term
`max_df`	1.0	Maximum document frequency (or fraction) to exclude a term
`ngram_range`	(1,1)	Include unigrams, bigrams, etc.
`sublinear_tf`	False	Use $\log(1 + \mathrm{tf})$ instead of raw tf
`norm`	'l2'	Normalization of the final vector
`smooth_idf`	True	IDF smoothing

Tip: `min_df=2` or `min_df=0.01` reduces vocabulary size considerably and removes noisy terms

TF-IDF vs CountVectorizer: Quick Comparison

```
from sklearn.feature_extraction.text import (
    CountVectorizer, TfidfVectorizer
)
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score

for Vectorizer in [CountVectorizer, TfidfVectorizer]:
    pipe = Pipeline([
        ('vec', Vectorizer(max_features=10000)),
        ('clf', LogisticRegression(max_iter=1000))
    ])
    scores = cross_val_score(pipe, X_train, y_train, cv=5)
    print(f"{Vectorizer.__name__}: {scores.mean():.3f}")
```

Application: Sentiment Analysis on IMDb

The IMDb Reviews Dataset

- 50,000 movie reviews
- Binary labels: **positive** / **negative**
- 25,000 train, 25,000 test
- Balanced (50/50)

"This film was absolutely wonderful. The acting was superb and the story kept me engaged throughout." $\rightarrow$ **positive**

"Terrible plot, wooden acting, and a complete waste of two hours." $\rightarrow$ **negative**

Loading the dataset

```
from datasets import load_dataset

dataset = load_dataset("imdb")
train_texts = dataset["train"]["text"]
train_labels = dataset["train"]["label"]
test_texts   = dataset["test"]["text"]
test_labels  = dataset["test"]["label"]

print(f"Train: {len(train_texts)} | Test: {len(test_texts)}")
print(f"Example:\n{train_texts[0][:200]}")
print(f"Label: {'pos' if train_labels[0] else 'neg'}")
```

Baseline pipeline: TF-IDF + Logistic Regression

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report

pipe = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=20000,
                              sublinear_tf=True)),
    ('clf',   LogisticRegression(C=1.0, max_iter=1000))
])

pipe.fit(train_texts, train_labels)
preds = pipe.predict(test_texts)

print(f"Accuracy: {accuracy_score(test_labels, preds):.4f}")
print(classification_report(test_labels, preds,
                            target_names=['neg', 'pos']))
```

Some experiments

Experiment	Configuration	What do we observe?
Baseline	CountVectorizer	Reference score
TF-IDF	TfidfVectorizer	Does it improve?
Sublinear	`sublinear_tf=True`	Effect of log(TF)
Bigrams	`ngram_range=(1,2)`	Does context help?
Vocab trim	`min_df=5, max_df=0.9`	Cleaner vocabulary

Interpretability: most influential words

One advantage of TF-IDF + logistic regression: the coefficients are interpretable.

Positive words

excellent, outstanding, brilliant, wonderful, superb, fascinating, masterpiece, ...

Negative words

awful, terrible, worst, waste, boring, dreadful, ridiculous, disappointing, ...

```
feature_names = pipe['tfidf'].get_feature_names_out()
coefs = pipe['clf'].coef_[0]
top_pos = feature_names[coefs.argsort()[-15:]]
top_neg = feature_names[coefs.argsort()[:15]]
```

Limitations and next steps

What does TF-IDF miss?

- **Word order**: "good, not bad" = "not good, bad"
- **Synonymy**: "car" and "automobile" are different dimensions
- **Polysemy**: "bank" (river vs. finance)
- **Long-range context**: no memory beyond the n-gram window

Alternatives to TF-IDF

- **Word embeddings** (Word2Vec, GloVe): capture semantic similarity
- **Transformers** (BERT, etc.): bidirectional context

When is TF-IDF still a good choice?

Despite its limitations, TF-IDF remains a solid option when:

- The dataset is **large** and embeddings are too costly to compute
- **Interpretability** is required (audit, regulation)
- The task is **purely topic-based** (routing, domain classification)
- It is used as a **baseline** before more complex models
- The domain is **highly specialized** and pretrained models do not help

Summary

- **BoW** ignores the fact that not all words are equally informative
- **TF** rewards words that are frequent in a document
- **IDF** penalises words that are frequent across the whole corpus
- **TF-IDF** = TF $\times$ IDF, with L2 normalization of the vector
- **scikit-learn**: `TfidfVectorizer` does all of this in one line
- In sentiment analysis, TF-IDF captures lexical signals well
- Its limitations (word order, semantics) motivate the move to **embeddings**